

# Postmortem: Cascading Compute and Database Failure

**Date:** May 3, 2026

**Author:** Jiehui Li

**Status:** Complete

## Summary

**Duration:** 15:14 - 15:45 (31 minutes)

**Impact:** Synthetic curl traffic targeting the ALB generated HTTP 500 errors during the fault window as the ALB attempted to route requests to the terminated EC2 instance. Error rate peaked at 13.57% on the dashboard, significantly breaching the 1.0% SLO error budget. Direct browser verification of the user-facing experience was not performed during the fault window.

**Root Cause:** Abrupt termination of the Auto Scaling EC2 instance left stale MySQL connections open, exhausting the `db.t3.micro` connection limit (~66 max) and preventing the replacement instance from executing any database queries.

## Timeline

*(All times local - US Eastern)*

| Time   | Event                                                                                                           |
|--------|-----------------------------------------------------------------------------------------------------------------|
| 15:14  | Fault injected. Auto Scaling Instance <code>i-080d88a8e860f23ab</code> terminated                               |
| ~15:15 | Error rate spikes to 13.57% on dashboard, breaching 1.0% SLO line                                               |
| ~15:16 | ASG launches replacement instance <code>i-00a4a39e99fae1ce5</code> — <code>HealthyHostCount</code> remains at 1 |
| ~15:17 | Error rate returns to 0, the compute layer appears to have recovered                                            |
| 15:22  | <code>SLO-Breach-5XX-Error-Rate</code> alarm fires, SNS email received                                          |
| ~15:30 | Manual check reveals that <code>/students</code> still returning the error page; investigation began            |

| Time   | Event                                                                                                 |
|--------|-------------------------------------------------------------------------------------------------------|
| ~15:31 | Direct MySQL connection returns <code>ERROR 1040: Too many connections</code> , root cause identified |
| ~15:32 | Mitigation applied, <code>CapstoneDB</code> rebooted to clear stale connections                       |
| ~15:45 | RDS available, <code>/students</code> confirmed working, the system is fully recovered                |

## Root Cause Analysis

When the EC2 instance was terminated under active load, the Node.js MySQL connection pool did not close connections gracefully. MySQL retained these stale connections server-side until timeout (default 8 hours), while the replacement ASG instance and ongoing curl traffic simultaneously opened new connections, collectively exhausting the `db.t3.micro` limit of ~66 connections. The ALB health check path `/` has no database dependency, so the EC2 instance continued reporting healthy while every `/students` request silently failed with `ERROR 1040: Too many connections`.

## Detection

**How detected:** Initial error rate spike detected visually on dashboard at ~15:15. `SL0-Breach-5XX-Error-Rate` alarm fired at 15:22. Secondary database failure detected only via manual browser test at ~15:30, after the dashboard showed apparent recovery. No alarm fired for this phase of the incident.

### TTD:

- Dashboard visual: **~1 minute**
- Alarm notification: **8 minutes**
- Secondary failure: **~16 minutes (manual only)**

**Was detection adequate?** Partially. The dashboard captured the initial error spike within 1 minute. However, the alarm fired 6 minutes after apparent recovery, providing no actionable response value. The database connection exhaustion, responsible for the majority of user impact, generated zero monitoring signals and was discovered only through manual testing by visiting `/students` site.

**What would improve detection?** A synthetic probe hitting `/students` every 60 seconds would have detected the secondary failure within 1 minute. A

`HealthyHostCount < 1` alarm with a 1-minute period would improve compute failure detection. An RDS `DatabaseConnections < 1` alarm would independently surface database connectivity loss.

## Response and Recovery

### Diagnostic steps:

1. Observed error rate spike on dashboard, identified compute failure
2. Confirmed ASG replacement instance is healthy in `CapstoneTG`
3. Manually tested `/students` , discovered the error page is still active
4. Connected to ASG instance via EC2 Instance Connect, confirmed Node.js running
5. Verified Secrets Manager returning correct RDS endpoint
6. Ran a direct MySQL connection, received `ERROR 1040: Too many connections`
7. Checked RDS console, confirmed 59 active connections against ~66 max limit

### Recovery steps:

1. Navigated to **RDS** → `CapstoneDB` → **Actions** → **Reboot**
2. Waited for RDS status to return to Available
3. Confirmed `/students` loading correctly in the browser

### Were runbooks useful?

No runbook existed for `ERROR 1040` diagnosis, adding approximately 10 minutes of unstructured investigation.

**TTM:** 31 minutes (15:14 → 15:45)

**Was the response adequate?** The compute recovery was automatic and fast. However, the absence of database monitoring signals and a documented runbook for connection exhaustion meant the secondary failure was diagnosed through manual trial and error, significantly extending the incident.

## What Went Well

- ASG automatically replaced the terminated instance within ~2 minutes without manual intervention
- `HealthyHostCount` never dropped to 0, replacement was fast enough to maintain continuous ALB registration
- The Dashboard Error Rate widget clearly showed the spike within 1 minute of injection
- `SL0-Breach-5XX-Error-Rate` alarm correctly identified the breach and delivered an SNS notification
- Secrets Manager correctly provided RDS credentials to the replacement instance automatically
- EC2 Instance Connect provided immediate access for diagnosis without key management

## What Went Wrong

- Alarm fired 8 minutes after injection and 6 minutes after apparent recovery, too late for an actionable response
- No alarm existed for RDS connection count, database connection exhaustion was completely invisible to monitoring
- ALB health check path `/` has no database dependency, a healthy EC2 instance masked a complete database failure
- Dashboard showed `HealthyHostCount: 1` throughout despite `/students` being broken for 31 minutes
- No runbook existed for `ERROR 1040` diagnosis, adding ~10 minutes of unstructured investigation
- The curl traffic generator contributed to connection exhaustion and extended recovery time
- `SL0-Breach-p99-Latency` never fired despite service degradation, failed requests produce no latency data

## Action Items

| Action                                                                                                | Type    | Priority | Status |
|-------------------------------------------------------------------------------------------------------|---------|----------|--------|
| Add synthetic probe hitting <code>/students</code> every 60 seconds with alarm on non-200 response    | Detect  | Critical | TODO   |
| Add <code>HealthyHostCount &lt; 1</code> alarm with 1-minute evaluation period                        | Detect  | High     | TODO   |
| Add RDS <code>DatabaseConnections &lt; 1</code> alarm with SNS notification                           | Detect  | High     | TODO   |
| Reduce <code>SL0-Breach-5XX-Error-Rate</code> evaluation period from 5 min to 1 min                   | Detect  | High     | TODO   |
| Configure MySQL <code>wait_timeout</code> on RDS parameter group to reduce stale connection retention | Prevent | High     | TODO   |
| Update ALB health check path from <code>/</code> to <code>/students</code>                            | Prevent | Medium   | TODO   |
| Write runbook for <code>ERROR 1040</code> covering diagnosis commands and recovery steps              | Process | High     | TODO   |
| Update on-call handoff document to include RDS connection exhaustion as known failure mode            | Process | Medium   | TODO   |
| Establish protocol to stop synthetic traffic generators before executing recovery procedures          | Process | Medium   | TODO   |

## Lessons Learned

A single compute fault can cascade into a multi-layer failure through indirect resource exhaustion, a pattern common in production but rarely anticipated in monitoring design. The most significant lesson is that an ALB-centric observability stack is insufficient for a multi-tier architecture: the database layer requires independent monitoring coverage. The 31-minute incident was largely driven not by the original fault, but by the gap between what the dashboard showed (healthy) and what users experienced (broken). Defense in depth in observability is as important as defense in depth in infrastructure.

## Fault 2 Brief Summary (RDS Reboot)

**What was broken and why chosen:** `CapstoneDB` was rebooted to simulate a database restart event and test whether database-layer failures were detectable by the existing monitoring stack.

**TTD:** Never detected. No alarm fired. No dashboard metric spiked. The only observable signal was a minor dip in Traffic Volume, insufficient to trigger any alert.

**TTM:** ~18 minutes (16:05 → ~16:23) across three RDS reboots. The curl loop was stopped before the third reboot to allow clean connection recovery.

**What detection caught:** Nothing. The Node.js connection pool auto-reconnected silently after the first reboot, resulting in 0 HTTP 500s at the ALB. The `/students` error page was discovered only through manual browser testing. Recurring connection exhaustion driven by the still-running curl loop extended recovery and was also entirely invisible to monitoring.

**Recovery steps:** Three RDS reboots were required. The first reboot completed silently with auto-reconnection, but `/students` remained broken due to residual connection exhaustion. The second reboot briefly restored service, but the still-running curl loop immediately re-exhausted connections. The curl loop was stopped before the third reboot, allowing RDS to come back with a clean connection count. Full recovery confirmed at ~16:23.

**Key takeaway:** A complete database restart is entirely invisible to an ALB-centric monitoring stack when the application connection pool handles reconnection gracefully. The only reliable fix is database-layer monitoring independent of HTTP metrics, specifically RDS `DatabaseConnections` alarms and synthetic probes that validate end-to-end query execution rather than just HTTP reachability.